Product Requirements Document (PRD)

Project Name: CampusTrade (Temporary Title) **Project Type:** Peer-to-Peer Micro-Rental and Barter E-commerce Platform **Target Users:** University Students **Document Version:** 1.0

1. Executive Summary & Vision

The Problem: University students frequently require expensive, specialized items for short durations (e.g., microcontrollers for a hackathon, specific textbooks for a semester, formal wear for an event, or specialized tools). Purchasing these items outright is economically inefficient and creates waste. Furthermore, students often possess valuable skills but lack liquid capital.

The Solution: A localized, peer-to-peer micro-rental marketplace restricted to the campus ecosystem. It allows students to rent items for short, specific timeframes or bypass fiat currency entirely by trading skills and services via a structured barter system.

Objective: To deliver a functional, secure web application that handles user verification, calendar-based booking, dual-mode checkout (currency vs. barter), and real-time user negotiation. This project will demonstrate full-stack capabilities, complex relational database management, and real-time data handling.

2. Target Audience & Personas

- **The Resource Seeker (Borrower):** Needs quick access to project hardware, study solutions, or event gear without paying full retail price. Might prefer trading their time/skills if they are low on funds.
- **The Campus Hustler (Lender):** Has a dorm full of useful gear (calculators, gaming consoles, tools) and wants to monetize them or trade for academic/technical help from peers.

3. Core Features & Functional Requirements

A. Campus-Restricted Authentication

- **Requirement:** Ensure the platform remains a high-trust environment by restricting access to active students.
- **Implementation:** Account creation must be verified by a standard university email domain (e.g., `.edu` or specific college domains).

B. Dual-Mode Listing Engine

- **Requirement:** Sellers must be able to list items with flexible compensation models.
- **Implementation:** * Users define a daily/hourly fiat rental price.
  - Users can attach a "Barter Tag" detailing acceptable service trades (e.g., "Will waive a 2-day rental fee for 1 hour of React.js tutoring").

C. Temporal Booking Checkout

- **Requirement:** The system must prevent double-booking of physical items.
- **Implementation:** An integrated calendar UI. When a borrower selects dates, the backend queries the database for existing active rentals on that specific `item_id`. Unavailable dates are locked/greyed out on the frontend.

D. The Barter Negotiation Room

- **Requirement:** If a user selects "Propose Barter" at checkout, the system must facilitate a structured agreement.
- **Implementation:** A real-time chat instance opens between the lender and borrower. Once terms are agreed upon, the lender clicks an "Accept Trade" button, changing the transaction state to `Active` without requiring a standard payment gateway.

4. System Architecture & Technical Stack

The application will follow a modern client-server architecture, utilizing a RESTful API to manage the flow of data between the front-end user interfaces and the backend database.

- **Frontend:** React.js (or Next.js) - Chosen for robust state management during the complex checkout and calendar flows. Tailwind CSS for rapid, responsive UI styling.
- **Backend:** Node.js with Express.js - Fast, asynchronous event handling perfect for API routing and booking logic.
- **Database:** PostgreSQL - A relational database is mandatory for this project to strictly map the relationships between Users, Items, and time-based Bookings.
- **Real-time Communication:** Socket.io - Required for the Barter Negotiation chat room to function in real-time.
- **File Storage:** AWS S3 or Cloudinary - For handling and storing user-uploaded images of their items.

5. Database Schema (Entity Relationship)

To support temporal rentals and dual-checkouts, the relational schema is structured as follows:

Table Name	Primary Key	Foreign Keys	Key Attributes
Users	`user_id`	None	`name`, `edu_email`, `password_hash`, `trust_score`, `created_at`
Items	`item_id`	`owner_id` (Users)	`title`, `description`, `daily_price`, `accepts_barter` (Boolean), `barter_description`

Bookings	`booking_id`	`item_id` (Items), `borrower_id` (Users)	`start_date`, `end_date`, `status` (Pending/Active/Completed), `payment_type` (Fiat/Barter)
Reviews	`review_id`	`booking_id` (Bookings), `reviewer_id` (Users)	`rating` (1-5), `comment`, `created_at`

6. Algorithmic Complexity: Trust Scoring

To demonstrate advanced logic, the platform implements a dynamic Trust Score for users based on their transaction history.

The algorithm calculates a user's reliability (T_{score}) out of 100, weighting successful item returns against peer reviews:

$$T_{score} = \left(W_1 \times \frac{R_{positive}}{R_{total}}\right) + \left(W_2 \times \frac{L_{completed}}{L_{total}}\right) - P_{penalties}$$

Where:

- R represents the user's peer reviews.
- L represents the number of micro-loans/rentals.
- W_1 and W_2 are assigned weights (e.g., 40 and 60 respectively).
- $P_{penalties}$ is a fixed point deduction for late returns or damaged items flagged in the system.

7. Project Milestones & Development Phases

1. **Phase 1: Foundation (Weeks 1-2)**
    - Database schema setup and connection.
    - User authentication (edu email restriction).
    - Basic CRUD operations for user profiles and item listings.
2. **Phase 2: The Core Engine (Weeks 3-4)**
    - Implementation of the calendar booking logic.
    - Date validation to prevent overlapping bookings.
    - Standard checkout flow (simulated fiat payments).
3. **Phase 3: The Unique Hook (Weeks 5-6)**
    - Integration of WebSockets (Socket.io).
    - Building the real-time Barter Negotiation Room.
    - State management for accepting a barter trade vs. paying.
4. **Phase 4: Polish & Delivery (Weeks 7-8)**

- Implementation of the Trust Score algorithm.
- UI/UX refinement and responsive design testing.